

USE OF BIG-O NOTATION IN THE TIME COMPLEXITY ANALYSIS OF ALGORITHMS

Course Title : CSE 106

Section : 7

Faculty : Dr Mohammad Salah Uddin

Associate Professor

Department of Computer Science and Engineering

INTRODUCTION TO BIG-O NOTATION

 **Big-O notation describes the upper limit of an algorithm's performance.**

 **It measures scalability as input size grows.**

RULES

Big-O focuses on:

 **The worst case scenario**

 **Ignoring constant factors**

COMMON BIG-O NOTATIONS

 $O(1)$: Constant Time

 $O(\sqrt{n})$: Square Root Time

 $O(n)$: Linear Time

 $O(n^2)$: Quadric Time

 $O(\log n)$: Logarithmic Time

 $O(2^n)$: Exponential Time

 $O(n \log n)$: Linearithmic Time

 $O(n!)$: Factorial Growth Time

ETC

CONSTANT TIME COMPLEXITY $O(1)$

A constant time operation doesn't depend on the input size

```
1  #include <stdio.h>
2  int main() {
3      int arr[] = {10, 20, 30, 40, 50};
4      int value = arr[2];
5      printf("Value: %d\n", value);
6      return 0;
7  }
```

Let's analyze each part of the code:

1. Array Initialization: Initializing an array of fixed size (5 elements) is a constant time operation, $O(1)$.
2. Accessing an Element: Accessing an element in an array by index is a constant time operation, $O(1)$.
3. Printing a Value: Printing a single value is a constant time operation, $O(1)$.
4. Return Statement: Returning from the main function is a constant time operation, $O(1)$.

Since each (T_i) is $O(1)$: $[T = O(1) + O(1) + O(1) + O(1)]$

Combining these constant time complexities, we get: $[T = O(1)]$

Therefore, the overall time complexity of the code is $O(1)$

$*(T_i)$ represents the time complexity of the i -th operation

LINEAR TIME COMPLEXITY $O(N)$

let's analyze each part of the code:

1. Loop Initialization: The loop runs from $i = 0$ to $i < n$. This means the loop will execute n times.
2. Statements inside the loop: The `printf` function is called in each iteration of the loop. Assuming `printf` takes constant time, $O(1)$, the time complexity of this statement is $O(1)$.

Let's denote the time taken by the `printf` statement inside the loop as $T_s = O(1)$

The total time complexity (T) of the loop is the sum of the time complexities of each iteration: $[T = T_s + T_s + \dots + T_s]$ (n times).

Since each T_s is $O(1)$: $[T = O(1) + O(1) + \dots + O(1)]$ (n times).

Combining these constant time complexities, we get: $[T = n * O(1)]$

Therefore, the overall time complexity of the loop is $O(n)$

*(T_s) represents the time complexity of the statement

QUADRIC TIME COMPLEXITY $O(N^2)$



```
1  #include <stdio.h>
2  int main()
3  {
4      for (int i = 0; i < n; i++)
5      {
6          for (int j = 0; j < n; j++)
7          {
8              printf("%d %d\n", i, j);
9          }
10     }
11 }
```

let's analyze each part of the code:

1. Outer Loop: The outer loop runs from $i = 0$ to $i < n$. This means the outer loop will execute n times.
2. Inner Loop: For each iteration of the outer loop, the inner loop runs from $j = 0$ to $j < n$. This means the inner loop will also execute n times for each iteration of the outer loop.
3. Statements inside the inner loop: The `printf` function is called in each iteration of the inner loop. Assuming `printf` takes constant time, $O(1)$, the time complexity of this statement is $O(1)$.

QUADRIC TIME COMPLEXITY $O(N^2)$

Let's denote the time taken by the printf statement inside the inner loop as,

$$T_s = O(1)$$

The total time complexity (T) of the nested loops is the sum of the time.

```

1  #include <stdio.h>
2  int main()
3  {
4      for (int i = 0; i < n; i++)
5      {
6          for (int j = 0; j < n; j++)
7          {
8              printf("%d %d\n", i, j);
9          }
10     }
11 }
```

i	j	Number of Time
0	0	0
1	0	1
	1	
2	0	
	1	
	2	

i	j	Number of Time
n	0	
	1	
	2	
	⋮	
	n	

QUADRIC TIME COMPLEXITY $O(N^2)$



```
1  #include <stdio.h>
2  int main()
3  {
4      for (int i = 0; i < n; i++)
5      {
6          for (int j = 0; j < n; j++)
7          {
8              printf("%d %d\n", i, j);
9          }
10     }
11 }
```

we know that,

$$1+2+3+4+ \dots +n = \frac{n(n+1)}{2}$$
$$= \frac{n^2 + n}{2}$$

In this function, degree is n^2 . So, the time complexity of this function is $O(n^2)$

SQUARE ROOT TIME COMPLEXITY $O(\sqrt{N})$

```
1  #include <stdio.h>
2  int main()
3  {
4      int p = 0;
5      for (int i = 1; p <= n; i++)
6      {
7          p = p + i;
8      }
9  }
```

let's analyze each part of the code:

1. Initialization: Initializing the variable p is a constant time operation, $O(1)$.
2. Loop Condition and Increment: The loop runs as long as p is less than or equal to n . In each iteration, i is increased by 1 and p is updated by adding i to it.
3. Statements inside the loop: The statement inside the loop updates the value of p by adding i to it. This is a constant time operation, $O(1)$.

SQUARE ROOT TIME COMPLEXITY $O(\sqrt{n})$

Let's denote the time taken by the statement inside the loop as $T_s = O(1)$.
The loop runs until the sum of the first i natural numbers exceeds n .

```

1  #include <stdio.h>
2  int main()
3  {
4      int p = 0;
5      for (int i = 1; p <= n; i++)
6      {
7          p = p + i;
8      }
9  }

```

i	p
1	0+1
2	0+1
3	0+1+2+3
4	0+1+2+3+4
⋮	
k	0+1+2+3+4+.....+k

The loop will be stop when $p > n$.

$$\text{since, } p = \frac{k(k+1)}{2}$$

$$= \frac{k^2 + k}{2}$$

$$\text{so, } \frac{k^2 + k}{2} > n$$

$$\rightarrow k^2 > n$$

$$\rightarrow k > \sqrt{n}$$

Therefore, the loop runs approximately $\sqrt{n}$ times, which is $O(\sqrt{n})$.

LOGARITHMIC TIME COMPLEXITY $O(\log n)$



```
1  #include <stdio.h>
2  int main()
3  {
4      for (int i = 1; i < n; i = i * 2)
5      {
6          printf("%d\n", i);
7      }
8  }
```

let's analyze each part of the code:

1. **Loop Initialization:** The loop starts with $i = 1$ and runs as long as i is less than n . In each iteration, i is multiplied by 2.
2. **Statements inside the loop:** The `printf` function is called in each iteration of the loop. Assuming `printf` takes constant time, $O(1)$, the time complexity of this statement is $O(1)$.

LOGARITHMIC TIME COMPLEXITY $O(\log n)$



```
1  #include <stdio.h>
2  int main()
3  {
4      for (int i = 1; i < n; i = i * 2)
5      {
6          printf("%d\n", i);
7      }
8  }
```

i
1
$1*2=2$
$1*2*2=2^2$
$1*2*2*2=2^3$
...
2^k

The loop runs with i taking values 1, 2, 4, 8, ..., up to the largest power of 2 less than n .

The loop will be terminated when $i \leq n$

Since, $i = 2^k$

$2^k \geq n$

$2^k = n$

$\log(2^k) = \log(n)$

$k * \log_2(2) = \log_2(n)$

$k = \log_2(n)$

Therefore, the loop runs approximately $\log_2(n)$ times, which is $O(\log n)$

LINEARITHMIC TIME COMPLEXITY $O(N \log N)$



```
1  #include <stdio.h>
2  int main()
3  {
4      for (int i = 1; i < n; i++)
5      {
6          for (int j = 1; j < n; j = j * 2)
7          {
8              printf("%d %d\n", i, j);
9          }
10     }
11 }
```

let's analyze each part of the code:

1. Outer Loop: The outer loop runs from $i = 1$ to $i < n$. This means the outer loop will execute $n - 1$ times, which is $O(n)$.
2. Inner Loop: For each iteration of the outer loop, the inner loop runs from $j = 1$ to $j < n$, with j being multiplied by 2 in each iteration. This means the inner loop will execute approximately $\log_2(n)$ times, which is $O(\log n)$.
3. Statements inside the inner loop: The printf function is called in each iteration of the inner loop. Assuming printf takes constant time, $O(1)$, the time complexity of this statement is $O(1)$.

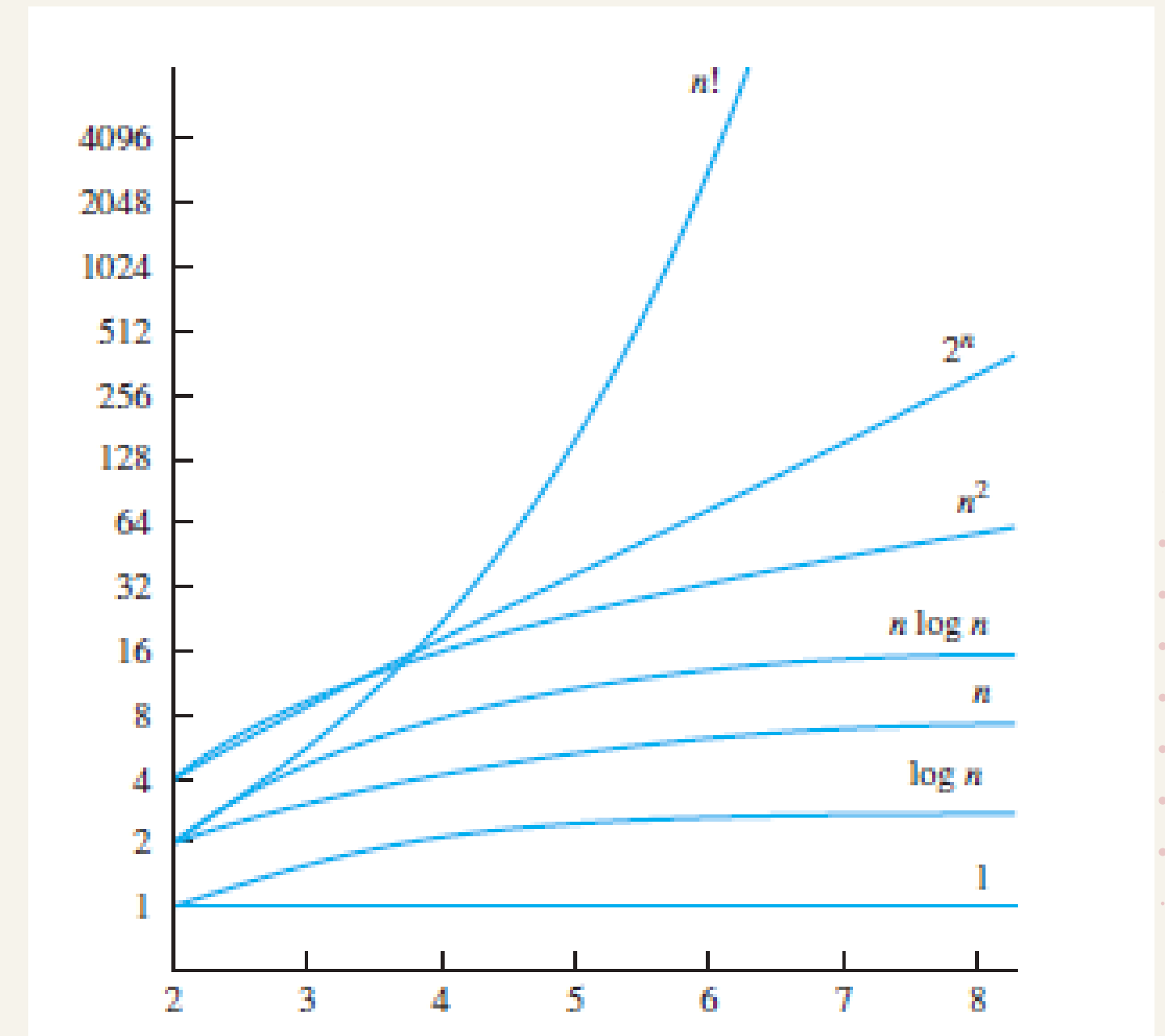
This can be simplified to: $T = (n - 1) * (\log n) * 1$

$\rightarrow T = n * (\log n)$

Therefore, the overall time complexity of the nested loops is $O(n \log n)$.

GRAPH

The graph visually demonstrates how different Big O complexities behave as input size increases, helping to understand their relative performance and efficiency.



BUBBLE SORT ALGORITHM

```
1  #include <stdio.h>
2  int main()
3  {
4      int arr[] = {64, 34, 25, 12, 22, 11, 90};
5      int n = sizeof(arr) / sizeof(arr[0]);
6      for (int i = 0; i < n - 1; i++)
7      {
8          for (int j = 0; j < n - i - 1; j++)
9          {
10             if (arr[j] > arr[j + 1])
11             {
12                 int temp = arr[j];
13                 arr[j] = arr[j + 1];
14                 arr[j + 1] = temp;
15             }
16         }
17     }
18     for (int i = 0; i < n; i++)
19     {
20         printf("%d ", arr[i]);
21     }
22
23     return 0;
24 }
```

let's analyze each part of the code:

1. **Outer Loop:** The outer loop runs from $i = 0$ to $i < n - 1$. This means the outer loop will execute $n - 1$ times, which is $O(n)$.
2. **Inner Loop:** For each iteration of the outer loop, the inner loop runs from $j = 0$ to $j < n - i - 1$. This means the inner loop will execute fewer times as i increases, but in the worst case, it will still execute approximately n times for each iteration of the outer loop.
3. **Comparison and Swap:** The comparison and swap operations inside the inner loop are constant time operations, $O(1)$.

This can be simplified to: $T = (n - 1) * O(n)$
 $= O(n^2)$ (Worst Case)

Therefore, the overall time complexity of the Bubble Sort algorithm is $O(n^2)$.

INSERTION SORT ALGORITHM

```
1  #include <stdio.h>
2  int main()
3  {
4      int arr[] = {64, 34, 25, 12, 22, 11, 90};
5      int n = sizeof(arr) / sizeof(arr[0]);
6      for (int i = 1; i < n; i++)
7      {
8          int key = arr[i];
9          int j = i - 1;
10         while (j >= 0 && arr[j] > key)
11         {
12             arr[j + 1] = arr[j];
13             j = j - 1;
14         }
15         arr[j + 1] = key;
16     }
17     for (int i = 0; i < n; i++)
18     {
19         printf("%d ", arr[i]);
20     }
21     return 0;
22 }
```

let's analyze each part of the code:

1. Outer Loop: The outer loop runs from $i = 1$ to $i < n$. This means the outer loop will execute n times, which is $O(n)$.
2. Inner Loop (while): For each iteration of the outer loop, the inner loop runs while j is greater than or equal to 0 and $arr[j]$ is greater than key . In the worst case, the inner loop will execute i times for each iteration of the outer loop.
3. Statements inside the inner loop: The statement in the inner loop are constant time operations, $O(1)$

This can be simplified to: $T = O(n^2)$ (Worst Case)

Therefore, the overall time complexity of the Insertion Sort algorithm is $O(n^2)$.

QUICK SORT ALGORITHM

```

1  #include <stdio.h>
2  void swap(int *a, int *b)
3  {
4      int temp = *a;
5      *a = *b;
6      *b = temp;
7  }
8  int partition(int arr[], int low, int high)
9  {
10     int pivot = arr[high];
11     int i = low - 1;
12     for (int j = low; j < high; j++)
13     {
14         if (arr[j] < pivot)
15         {
16             i++;
17             swap(&arr[i], &arr[j]);
18         }
19     }
20     swap(&arr[i + 1], &arr[high]);
21     return i + 1;
22 }

23
24 void quickSort(int arr[], int low, int high)
25 {
26     if (low < high)
27     {
28         int pi = partition(arr, low, high);
29         quickSort(arr, low, pi - 1);
30         quickSort(arr, pi + 1, high);
31     }
32 }
33
34 int main()
35 {
36     int arr[] = {10, 7, 8, 9, 1, 5};
37     int n = sizeof(arr) / sizeof(arr[0]);
38     quickSort(arr, 0, n - 1);
39     for (int i = 0; i < n; i++)
40     {
41         printf("%d ", arr[i]);
42     }
43     return 0;
44 }

```

Let's analyze each part of the code:

1. **Partition Function:** The partition function rearranges the elements in the array such that elements less than the pivot are on the left, and elements greater than the pivot are on the right. This function runs in $O(n)$ time, where n is the number of elements between low and high.
2. **Quick Sort Function:** The quickSort function recursively sorts the subarrays. The time complexity of quickSort depends on the partitioning of the array.

QUICK SORT ALGORITHM

```

1  #include <stdio.h>
2  void swap(int *a, int *b)
3  {
4      int temp = *a;
5      *a = *b;
6      *b = temp;
7  }
8  int partition(int arr[], int low, int high)
9  {
10     int pivot = arr[high];
11     int i = low - 1;
12     for (int j = low; j < high; j++)
13     {
14         if (arr[j] < pivot)
15         {
16             i++;
17             swap(&arr[i], &arr[j]);
18         }
19     }
20     swap(&arr[i + 1], &arr[high]);
21     return i + 1;
22 }
23
24 void quickSort(int arr[], int low, int high)
25 {
26     if (low < high)
27     {
28         int pi = partition(arr, low, high);
29         quickSort(arr, low, pi - 1);
30         quickSort(arr, pi + 1, high);
31     }
32 }
33
34 int main()
35 {
36     int arr[] = {10, 7, 8, 9, 1, 5};
37     int n = sizeof(arr) / sizeof(arr[0]);
38     quickSort(arr, 0, n - 1);
39     for (int i = 0; i < n; i++)
40     {
41         printf("%d ", arr[i]);
42     }
43     return 0;
44 }

```

In the worst case, the pivot is the smallest or largest element, resulting in one subarray with $n-1$ elements and the other with 0 elements. The recurrence relation for the worst case is: $T(n) = T(n-1) + O(n)$

This simplifies to: $T(n) = O(n^2)$

LINEAR SEARCH ALGORITHM

```
1  #include <stdio.h>
2  int main()
3  {
4      int arr[] = {10, 20, 30, 40, 50};
5      int n = sizeof(arr) / sizeof(arr[0]);
6      int x = 30;
7      for (int i = 0; i < n; i++)
8      {
9          if (arr[i] == x)
10         {
11             printf("Element found at index %d\n", i);
12             return 0;
13         }
14     }
15     printf("Element not found in the array\n");
16     return 0;
17 }
```

let's analyze each part of the code:

1. Initialization: Initializing the array, calculating its size, and setting the value of x are all constant time operations, $O(1)$.
2. Loop : The loop runs from $i = 0$ to $i < n$. This means the loop will execute n times in the worst case.
3. Condition inside the loop: The condition checks if the current element is equal to x. This is a constant time operation, $O(1)$.
4. Statements inside the condition: If the element is found, the printf statement and return are executed, both of which are constant time operations, $O(1)$.

This can be simplified to: $T = O(n)$ (Worst Case)

Therefore, the overall time complexity of the Linear Search algorithm is $O(n)$.

THANK YOU

Presented By : Abdullah Al Junaid

Department of Computer Science and Engineering

2024-3-60-082